

將 Video Intelligence API 與 Python 搭配使用

來源：<https://codelabs.developers.google.com/codelabs/cloud-video-intelligence-python3?hl=zh-tw>

步驟數：13

本教學課程將說明如何搭配 Python 使用 Video Intelligence API。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 環境設定](#)
4. [4. 影片範例](#)
5. [5. 偵測鏡頭轉換](#)
6. [6. 偵測標籤](#)
7. [7. 偵測煽情露骨的內容](#)
8. [8. 轉錄語音](#)
9. [9. 偵測及追蹤文字](#)
10. [10. 偵測並追蹤物件](#)
11. [11. 偵測及追蹤標誌](#)
12. [12. 偵測多個特徵](#)
13. [13. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽



[Video Intelligence API](#) 可讓您在應用程式中使用 Google 影片分析技術。

在本實驗室中，您將著重於使用 Python 搭配 Video Intelligence API。

課程內容

- 如何設定環境
- 如何設定 Python
- 如何偵測鏡頭轉換
- 如何偵測標籤
- 如何偵測煽情露骨內容
- 如何轉錄語音
- 如何偵測及追蹤文字
- 如何偵測及追蹤物件
- 如何偵測及追蹤標誌

軟硬體需求

- 具備 Google Cloud 專案
- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 熟悉 Python

問卷調查

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section follows, with a 'Project name' field containing 'my-kubernetes-codelab' (circled in blue). Below the name field is the 'Project ID' field, which contains 'my-kubernetes-codelab. It cannot be changed later.' (circled in blue) and an 'EDIT' link. Below the ID field is the 'Location' field with 'No organization' selected and a 'BROWSE' link. At the bottom are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

啟用 Cloud Shell

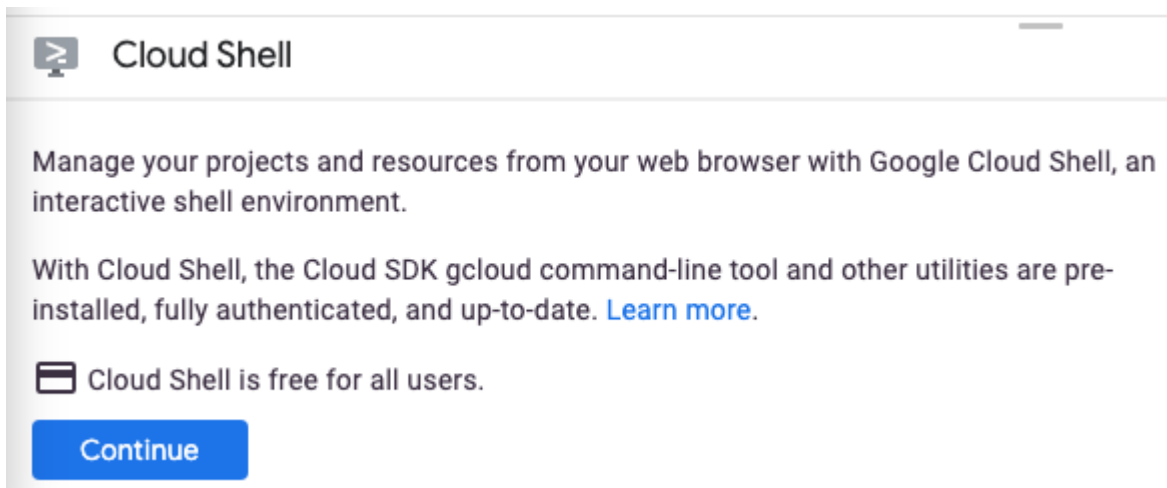
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



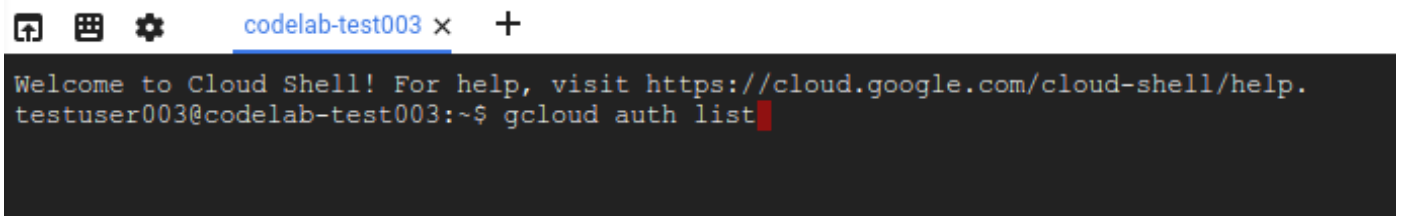
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

```
* <my_account>@<my_domain.com>
```

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 1 分鐘

3. 環境設定

如要開始使用 Video Intelligence API，請先在 Cloud Shell 執行下列指令來啟用 API：

```
gcloud services enable videointelligence.googleapis.com
```

畫面應如下所示：

```
Operation "operations/..." finished successfully.
```

現在您可以使用 Video Intelligence API 了！

前往主目錄：

```
cd ~
```

建立 Python 虛擬環境，隔離依附元件：

```
virtualenv venv-videointel
```

啟動虛擬環境：

```
source venv-videointel/bin/activate
```

注意：如要停止使用虛擬環境並返回系統 Python 版本，請使用 `deactivate` 指令。

安裝 IPython 和 Video Intelligence API 用戶端程式庫：

```
pip install ipython google-cloud-videointelligence
```

畫面應如下所示：

```
...  
Installing collected packages: ..., ipython, google-cloud-videointelligence  
Successfully installed ... google-cloud-videointelligence-2.11.0 ...
```

您現在可以使用 Video Intelligence API 用戶端程式庫了！

注意：如果您要在 Cloud Shell 以外的地方自行設定 Python 開發環境，請參閱這些[指南](#)。

在接下來的步驟中，您會使用名為 **IPython** 的互動式 Python 解譯器，這個解譯器已在先前的步驟中安裝。在 Cloud Shell 中執行 `ipython`，啟動工作階段：

```
ipython
```

畫面應如下所示：

```
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.12.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

注意：如有需要，您可以使用 `exit` 指令結束 IPython 工作階段。

步驟 4 · 約 0 分鐘

4. 影片範例

您可以使用 Video Intelligence API，為儲存在 Cloud Storage 中的影片加上註解，或以資料位元組的形式提供影片。

在後續步驟中，您將使用儲存在 Cloud Storage 中的範例影片。你也可以[在瀏覽器中觀看影片](#)。



預備，開始！

步驟 5 · 約 2 分鐘

5. 偵測鏡頭轉換

您可以使用 Video Intelligence API 偵測影片中的鏡頭轉換。鏡頭是影片片段，也就是一系列具有視覺連續性的影格。

將下列程式碼複製到 IPython 工作階段：

```
from typing import cast

from google.cloud import videointelligence_v1 as vi

def detect_shot_changes(video_uri: str) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.SHOT_CHANGE_DETECTION]
    request = vi.AnnotateVideoRequest(input_uri=video_uri, features=features)

    print(f'Processing video: "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `SHOT_CHANGE_DETECTION` 參數分析影片，並偵測鏡頭轉換。

注意：為方便本教學課程進行說明，您會同步等待作業完成。在正式環境中，您更有可能以非同步方式工作，在要求中定義 `output_uri` 欄位 (儲存結果的位置)，並在呼叫 `annotate_video` 後立即傳回。

呼叫函式來分析影片：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"

results = detect_shot_changes(video_uri)
```

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

如果收到 `PermissionDenied` 錯誤，請輸入 `exit` 退出 IPython，然後返回「設定和需求」步驟。

新增這個函式，即可列印影片鏡頭：

```
def print_video_shots(results: vi.VideoAnnotationResults):
    shots = results.shot_annotations
    print(f" Video shots: {len(shots)} ".center(40, "-"))
    for i, shot in enumerate(shots):
        t1 = shot.start_time_offset.total_seconds()
        t2 = shot.end_time_offset.total_seconds()
        print(f"{i+1:>3} | {t1:7.3f} | {t2:7.3f}")
```

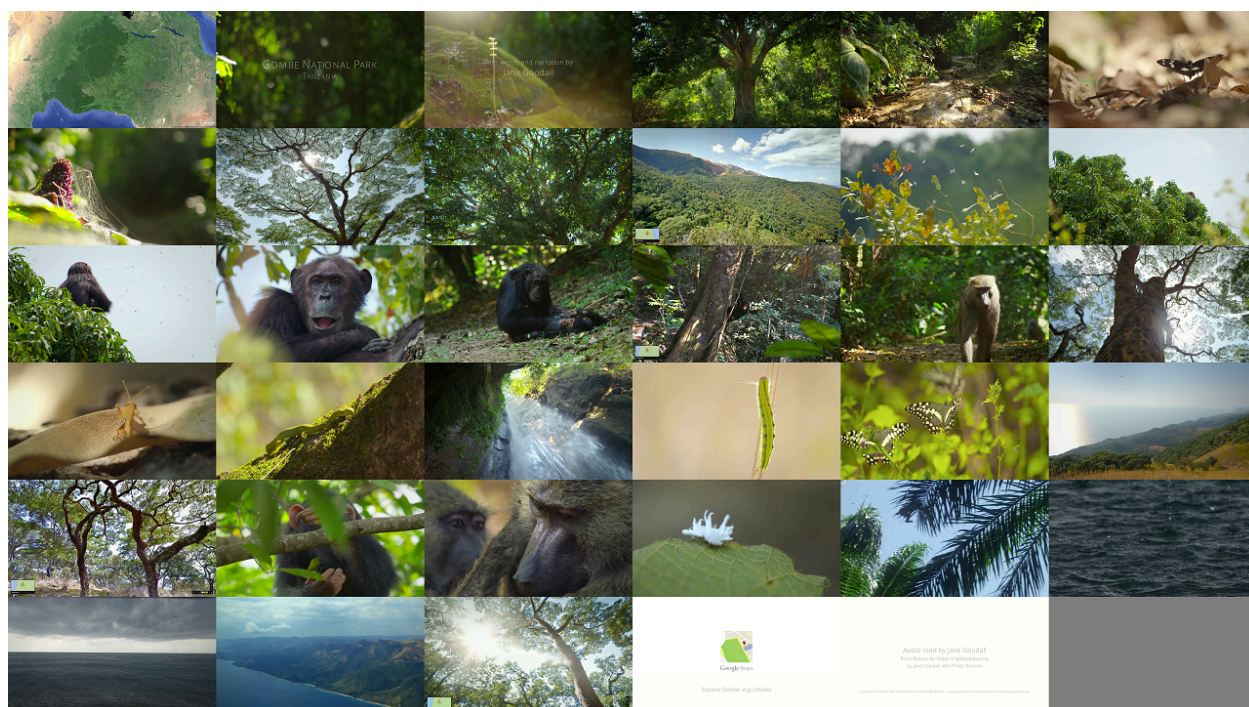
呼叫函式：

```
print_video_shots(results)
```

畫面應如下所示：

```
----- Video shots: 34 -----
  1 |   0.000 |  12.880
  2 |  12.920 |  21.680
  3 |  21.720 |  27.880
...
 32 | 135.160 | 138.320
 33 | 138.360 | 146.200
 34 | 146.240 | 162.520
```

如果擷取每個鏡頭的中間影格，並排列成影格牆，即可產生影片的視覺摘要：



摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行鏡頭轉換偵測。進一步瞭解如何[偵測鏡頭轉換](#)。

步驟 6 · 約 2 分鐘

6. 偵測標籤

您可以使用 Video Intelligence API 偵測影片中的標籤。標籤會根據影片的視覺內容描述影片。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def detect_labels(
    video_uri: str,
    mode: vi.LabelDetectionMode,
    segments: Optional[Sequence[vi.VideoSegment]] = None,
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.LABEL_DETECTION]
    config = vi.LabelDetectionConfig(label_detection_mode=mode)
    context = vi.VideoContext(segments=segments, label_detection_config=config)
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解程式碼如何使用 `annotate_video` 用戶端程式庫方法和 `LABEL_DETECTION` 參數分析影片及偵測標籤。

呼叫函式來分析影片的前 37 秒：

```

video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
mode = vi.LabelDetectionMode.SHOT_MODE
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=0),
    end_time_offset=timedelta(seconds=37),
)

results = detect_labels(video_uri, mode, [segment])

```

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這項函式，即可在影片層級列印標籤：

```

def print_video_labels(results: vi.VideoAnnotationResults):
    labels = sorted_by_first_segment_confidence(results.segment_label_annotations)

    print(f" Video labels: {len(labels)} ".center(80, "-"))
    for label in labels:
        categories = category_entities_to_str(label.category_entities)
        for segment in label.segments:
            confidence = segment.confidence
            t1 = segment.segment.start_time_offset.total_seconds()
            t2 = segment.segment.end_time_offset.total_seconds()
            print(
                f"{confidence:4.0%}",
                f"{t1:7.3f}",
                f"{t2:7.3f}",
                f"{label.entity.description}{categories}",
                sep=" | ",
            )

def sorted_by_first_segment_confidence(
    labels: Sequence[vi.LabelAnnotation],
) -> Sequence[vi.LabelAnnotation]:
    def first_segment_confidence(label: vi.LabelAnnotation) -> float:
        return label.segments[0].confidence

    return sorted(labels, key=first_segment_confidence, reverse=True)

def category_entities_to_str(category_entities: Sequence[vi.Entity]) -> str:
    if not category_entities:
        return ""
    entities = ", ".join([e.description for e in category_entities])
    return f" ({entities})"

```

呼叫函式：

```
print_video_labels(results)
```

畫面應如下所示：

```
----- Video labels: 10 -----
96% |    0.000 |   36.960 | nature
74% |    0.000 |   36.960 | vegetation
59% |    0.000 |   36.960 | tree (plant)
56% |    0.000 |   36.960 | forest (geographical feature)
49% |    0.000 |   36.960 | leaf (plant)
43% |    0.000 |   36.960 | flora (plant)
38% |    0.000 |   36.960 | nature reserve (geographical feature)
38% |    0.000 |   36.960 | woodland (forest)
35% |    0.000 |   36.960 | water resources (water)
32% |    0.000 |   36.960 | sunlight (light)
```

有了這些影片層級的標籤，您就能瞭解影片開頭主要與自然和植被相關。

新增這個函式，在鏡頭層級列印標籤：

```
def print_shot_labels(results: vi.VideoAnnotationResults):
    labels = sorted_by_first_segment_start_and_confidence(
        results.shot_label_annotations
    )

    print(f" Shot labels: {len(labels)} ".center(80, "-"))
    for label in labels:
        categories = category_entities_to_str(label.category_entities)
        print(f"{label.entity.description}{categories}")
        for segment in label.segments:
            confidence = segment.confidence
            t1 = segment.segment.start_time_offset.total_seconds()
            t2 = segment.segment.end_time_offset.total_seconds()
            print(f"{confidence:4.0%} | {t1:7.3f} | {t2:7.3f}")

def sorted_by_first_segment_start_and_confidence(
    labels: Sequence[vi.LabelAnnotation],
) -> Sequence[vi.LabelAnnotation]:
    def first_segment_start_and_confidence(label: vi.LabelAnnotation):
        first_segment = label.segments[0]
        ms = first_segment.segment.start_time_offset.total_seconds()
        return (ms, -first_segment.confidence)

    return sorted(labels, key=first_segment_start_and_confidence)
```

呼叫函式：

```
print_shot_labels(results)
```

畫面應如下所示：

```
----- Shot labels: 29 -----
planet (astronomical object)
 83% |   0.000 |  12.880
earth (planet)
 53% |   0.000 |  12.880
water resources (water)
 43% |   0.000 |  12.880
aerial photography (photography)
 43% |   0.000 |  12.880
vegetation
 32% |   0.000 |  12.880
 92% |  12.920 |  21.680
 83% |  21.720 |  27.880
 77% |  27.920 |  31.800
 76% |  31.840 |  34.720
...
butterfly (insect, animal)
 84% |  34.760 |  36.960
...
```

有了這些鏡頭層級的標籤，您就能瞭解影片開頭是行星 (可能是地球) 的鏡頭，`34.760-36.960s` 鏡頭中有一隻蝴蝶，等等。

注意：您也可以使用 `FRAME_MODE` 模式 (或 `SHOT_AND_FRAME_MODE`，同時使用兩種模式) 在影格層級要求標籤偵測。

摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行標籤偵測。如要進一步瞭解如何偵測標籤，請參閱[這篇文章](#)。

7. 偵測煽情露骨的內容

您可以使用 Video Intelligence API 偵測影片中的煽情露骨內容。露骨內容是一般不適合未滿 18 歲者的成人內容，包括但不限於裸露、性活動和色情內容。系統只會根據每格畫面的視覺信號進行偵測 (不會使用音訊)。回覆內容包含從 `VERY_UNLIKELY` 到 `VERY_LIKELY` 的可能性值。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def detect_explicit_content(
    video_uri: str,
    segments: Optional[Sequence[vi.VideoSegment]] = None,
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.EXPLICIT_CONTENT_DETECTION]
    context = vi.VideoContext(segments=segments)
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `EXPLICIT_CONTENT_DETECTION` 參數分析影片，並偵測煽情露骨內容。

呼叫函式來分析影片的前 10 秒：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=0),
    end_time_offset=timedelta(seconds=10),
)

results = detect_explicit_content(video_uri, [segment])
```

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這個函式，列印出不同的可能性計數：

```
def print_explicit_content(results: vi.VideoAnnotationResults):
    from collections import Counter

    frames = results.explicit_annotation.frames
    likelihood_counts = Counter([f.pornography_likelihood for f in frames])

    print(f" Explicit content frames: {len(frames)} ".center(40, "-"))
    for likelihood in vi.Likelihood:
        print(f"{likelihood.name:<22}: {likelihood_counts[likelihood]:>3}")
```

呼叫函式：

```
print_explicit_content(results)
```

畫面應如下所示：

```
----- Explicit content frames: 10 -----
LIKELIHOOD_UNSPECIFIED:    0
VERY_UNLIKELY             : 10
UNLIKELY                  :  0
POSSIBLE                  :  0
LIKELY                    :  0
VERY_LIKELY               :  0
```

新增這個函式，即可列印影格詳細資料：

```
def print_frames(results: vi.VideoAnnotationResults, likelihood: vi.Likelihood):
    frames = results.explicit_annotation.frames
    frames = [f for f in frames if f.pornography_likelihood == likelihood]

    print(f" {likelihood.name} frames: {len(frames)} ".center(40, "-"))
    for frame in frames:
        print(frame.time_offset)
```

呼叫函式：

```
print_frames(results, vi.Likelihood.VERY_UNLIKELY)
```

畫面應如下所示：

```
----- VERY_UNLIKELY frames: 10 -----  
0:00:00.365992  
0:00:01.279206  
0:00:02.268336  
0:00:03.289253  
0:00:04.400163  
0:00:05.291547  
0:00:06.449558  
0:00:07.452751  
0:00:08.577405  
0:00:09.554514
```

摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行煽情露骨內容偵測。如要進一步瞭解如何偵測露骨內容，請參閱[這篇文章](#)。

步驟 8 · 約 2 分鐘

8. 轉錄語音

您可以使用 Video Intelligence API 將影片中的語音轉錄為文字。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def transcribe_speech(
    video_uri: str,
    language_code: str,
    segments: Optional[Sequence[vi.VideoSegment]] = None,
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.SPEECH_TRANSCRIPTION]
    config = vi.SpeechTranscriptionConfig(
        language_code=language_code,
        enable_automatic_punctuation=True,
    )
    context = vi.VideoContext(
        segments=segments,
        speech_transcription_config=config,
    )
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `SPEECH_TRANSCRIPTION` 參數分析影片並轉錄語音。

呼叫函式，分析第 55 秒到第 80 秒的影片：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
language_code = "en-GB"
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=55),
    end_time_offset=timedelta(seconds=80),
)

results = transcribe_speech(video_uri, language_code, [segment])
```

注意：由於旁白員是英國人，`language_code` 會設為 `en-GB`。如需目前支援的語言代碼清單，請參閱[語言支援](#)。

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這個函式，即可輸出轉錄的語音：

```
def print_video_speech(results: vi.VideoAnnotationResults, min_confidence: float = 0.8):
    def keep_transcription(transcription: vi.SpeechTranscription) -> bool:
        return min_confidence <= transcription.alternatives[0].confidence

    transcriptions = results.speech_transcriptions
    transcriptions = [t for t in transcriptions if keep_transcription(t)]

    print(f" Speech transcriptions: {len(transcriptions)} ".center(80, "-"))
    for transcription in transcriptions:
        first_alternative = transcription.alternatives[0]
        confidence = first_alternative.confidence
        transcript = first_alternative.transcript
        print(f" {confidence:4.0%} | {transcript.strip()}")
```

呼叫函式：

```
print_video_speech(results)
```

畫面應如下所示：

```
----- Speech transcriptions: 2 -----
 91% | I was keenly aware of secret movements in the trees.
 92% | I looked into his large and lustrous eyes. They seem somehow to express his
entire personality.
```

新增這項函式，即可列印偵測到的字詞清單和時間戳記：

```
def print_word_timestamps(
    results: vi.VideoAnnotationResults,
    min_confidence: float = 0.8,
):
    def keep_transcription(transcription: vi.SpeechTranscription) -> bool:
        return min_confidence <= transcription.alternatives[0].confidence

    transcriptions = results.speech_transcriptions
    transcriptions = [t for t in transcriptions if keep_transcription(t)]

    print(" Word timestamps ".center(80, "-"))
    for transcription in transcriptions:
        first_alternative = transcription.alternatives[0]
        confidence = first_alternative.confidence
        for word in first_alternative.words:
            t1 = word.start_time.total_seconds()
            t2 = word.end_time.total_seconds()
            word = word.word
            print(f"{confidence:4.0%} | {t1:7.3f} | {t2:7.3f} | {word}")
```

呼叫函式：

```
print_word_timestamps(results)
```

畫面應如下所示：

```
----- Word timestamps -----
93% | 55.000 | 55.700 | I
93% | 55.700 | 55.900 | was
93% | 55.900 | 56.300 | keenly
93% | 56.300 | 56.700 | aware
93% | 56.700 | 56.900 | of
...
94% | 76.900 | 77.400 | express
94% | 77.400 | 77.600 | his
94% | 77.600 | 78.200 | entire
94% | 78.200 | 78.500 | personality.
```

摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行語音轉錄。如要進一步瞭解如何轉錄音訊，請參閱[這篇文章](#)。

9. 偵測及追蹤文字

您可以使用 Video Intelligence API 偵測及追蹤影片中的文字。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def detect_text(
    video_uri: str,
    language_hints: Optional[Sequence[str]] = None,
    segments: Optional[Sequence[vi.VideoSegment]] = None,
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.TEXT_DETECTION]
    config = vi.TextDetectionConfig(
        language_hints=language_hints,
    )
    context = vi.VideoContext(
        segments=segments,
        text_detection_config=config,
    )
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `TEXT_DETECTION` 參數分析影片及偵測文字。

呼叫函式，分析第 13 秒到第 27 秒的影片：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=13),
    end_time_offset=timedelta(seconds=27),
)

results = detect_text(video_uri, segments=[segment])
```

注意：系統預設會自動偵測語言。 `language_hints` (如果事先知道要偵測的語言)，可指定要偵測的語言，提高偵測準確度。

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這個函式，即可輸出偵測到的文字：

```

def print_video_text(results: vi.VideoAnnotationResults, min_frames: int = 15):
    annotations = sorted_by_first_segment_end(results.text_annotations)

    print(" Detected text ".center(80, "-"))
    for annotation in annotations:
        for text_segment in annotation.segments:
            frames = len(text_segment.frames)
            if frames < min_frames:
                continue
            text = annotation.text
            confidence = text_segment.confidence
            start = text_segment.segment.start_time_offset
            seconds = segment_seconds(text_segment.segment)
            print(text)
            print(f"  {confidence:4.0%} | {start} + {seconds:.1f}s | {frames} fr.")

def sorted_by_first_segment_end(
    annotations: Sequence[vi.TextAnnotation],
) -> Sequence[vi.TextAnnotation]:
    def first_segment_end(annotation: vi.TextAnnotation) -> int:
        return annotation.segments[0].segment.end_time_offset.total_seconds()

    return sorted(annotations, key=first_segment_end)

def segment_seconds(segment: vi.VideoSegment) -> float:
    t1 = segment.start_time_offset.total_seconds()
    t2 = segment.end_time_offset.total_seconds()
    return t2 - t1

```

呼叫函式：

```
print_video_text(results)
```

畫面應如下所示：

```

----- Detected text -----
GOMBE NATIONAL PARK
  99% | 0:00:15.760000 + 1.7s | 15 fr.
TANZANIA
 100% | 0:00:15.760000 + 4.8s | 39 fr.
With words and narration by
 100% | 0:00:23.200000 + 3.6s | 31 fr.
Jane Goodall
  99% | 0:00:23.080000 + 3.8s | 33 fr.

```

新增這個函式，列印偵測到的文字影格和周框：

```

def print_text_frames(results: vi.VideoAnnotationResults, contained_text: str):
    # Vertex order: top-left, top-right, bottom-right, bottom-left
    def box_top_left(box: vi.NormalizedBoundingPoly) -> str:
        tl = box.vertices[0]
        return f"({tl.x:.5f}, {tl.y:.5f})"

    def box_bottom_right(box: vi.NormalizedBoundingPoly) -> str:
        br = box.vertices[2]
        return f"({br.x:.5f}, {br.y:.5f})"

    annotations = results.text_annotations
    annotations = [a for a in annotations if contained_text in a.text]
    for annotation in annotations:
        print(f" {annotation.text} ".center(80, "-"))
        for text_segment in annotation.segments:
            for frame in text_segment.frames:
                frame_ms = frame.time_offset.total_seconds()
                box = frame.rotated_bounding_box
                print(
                    f"{frame_ms:>7.3f}",
                    box_top_left(box),
                    box_bottom_right(box),
                    sep=" | ",
                )

```

呼叫函式，檢查哪些影格會顯示旁白者的姓名：

```

contained_text = "Goodall"
print_text_frames(results, contained_text)

```

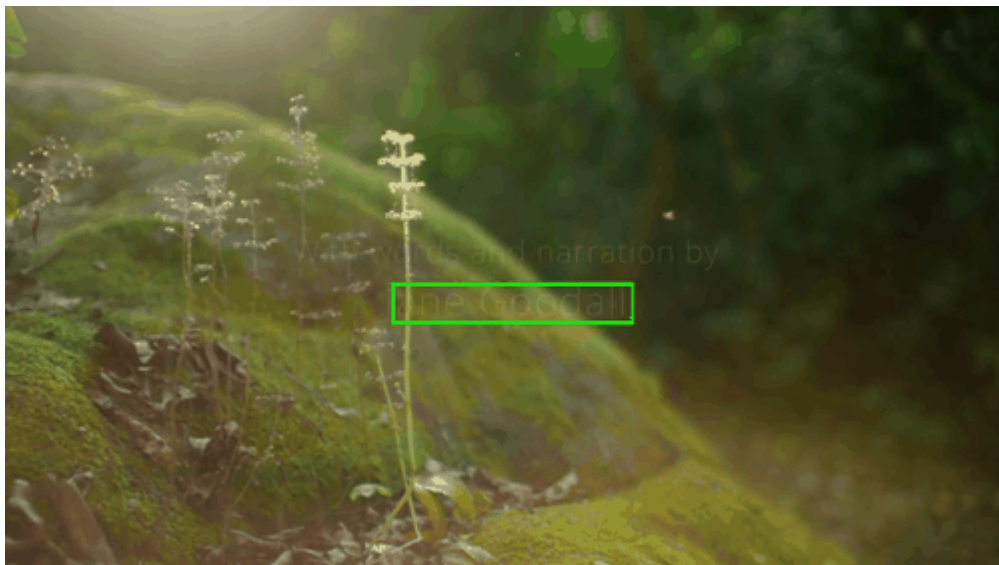
畫面應如下所示：

```

----- Jane Goodall -----
23.080 | (0.39922, 0.49861) | (0.62752, 0.55888)
23.200 | (0.38750, 0.49028) | (0.62692, 0.56306)
...
26.800 | (0.36016, 0.49583) | (0.61094, 0.56048)
26.920 | (0.45859, 0.49583) | (0.60365, 0.56174)

```

在對應影格上繪製定界框後，會得到以下結果：



摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行文字偵測和追蹤作業。進一步瞭解如何[偵測及追蹤文字](#)。

步驟 10 · 約 2 分鐘

10. 偵測並追蹤物件

您可以使用 Video Intelligence API 偵測及追蹤影片中的物件。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def track_objects(
    video_uri: str, segments: Optional[Sequence[vi.VideoSegment]] = None
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.OBJECT_TRACKING]
    context = vi.VideoContext(segments=segments)
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `OBJECT_TRACKING` 參數分析影片及偵測物件。

呼叫函式，分析影片第 98 秒到第 112 秒的內容：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=98),
    end_time_offset=timedelta(seconds=112),
)

results = track_objects(video_uri, [segment])
```

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這個函式，列印偵測到的物件清單：

```
def print_detected_objects(
    results: vi.VideoAnnotationResults,
    min_confidence: float = 0.7,
):
    annotations = results.object_annotations
    annotations = [a for a in annotations if min_confidence <= a.confidence]

    print(
        f" Detected objects: {len(annotations)}"
        f" ({min_confidence:.0%} <= confidence) ".center(80, "-")
    )
    for annotation in annotations:
        entity = annotation.entity
        description = entity.description
        entity_id = entity.entity_id
        confidence = annotation.confidence
        t1 = annotation.segment.start_time_offset.total_seconds()
        t2 = annotation.segment.end_time_offset.total_seconds()
        frames = len(annotation.frames)
        print(
            f"{description:<22}",
            f"{entity_id:<10}",
            f"{confidence:4.0%}",
            f"{t1:>7.3f}",
            f"{t2:>7.3f}",
            f"{frames:>2} fr.",
            sep=" | ",
        )
```

呼叫函式：

```
print_detected_objects(results)
```

畫面應如下所示：

```
----- Detected objects: 3 (70% <= confidence) -----
insect          | /m/03vt0 | 87% | 98.840 | 101.720 | 25 fr.
insect          | /m/03vt0 | 71% | 108.440 | 111.080 | 23 fr.
butterfly       | /m/0cyf8 | 91% | 111.200 | 111.920 | 7 fr.
```

新增這個函式，列印偵測到的物件影格和周框方塊清單：

```

def print_object_frames(
    results: vi.VideoAnnotationResults,
    entity_id: str,
    min_confidence: float = 0.7,
):
    def keep_annotation(annotation: vi.ObjectTrackingAnnotation) -> bool:
        return (
            annotation.entity.entity_id == entity_id
            and min_confidence <= annotation.confidence
        )

    annotations = results.object_annotations
    annotations = [a for a in annotations if keep_annotation(a)]
    for annotation in annotations:
        description = annotation.entity.description
        confidence = annotation.confidence
        print(
            f" {description}, "
            f" confidence: {confidence:.0%}, "
            f" frames: {len(annotation.frames)} " .center(80, "-")
        )
    for frame in annotation.frames:
        t = frame.time_offset.total_seconds()
        box = frame.normalized_bounding_box
        print(
            f"{t:>7.3f}",
            f"({box.left:.5f}, {box.top:.5f})",
            f"({box.right:.5f}, {box.bottom:.5f})",
            sep=" | ",
        )

```

使用昆蟲的實體 ID 呼叫函式：

```

insect_entity_id = "/m/03vt0"
print_object_frames(results, insect_entity_id)

```

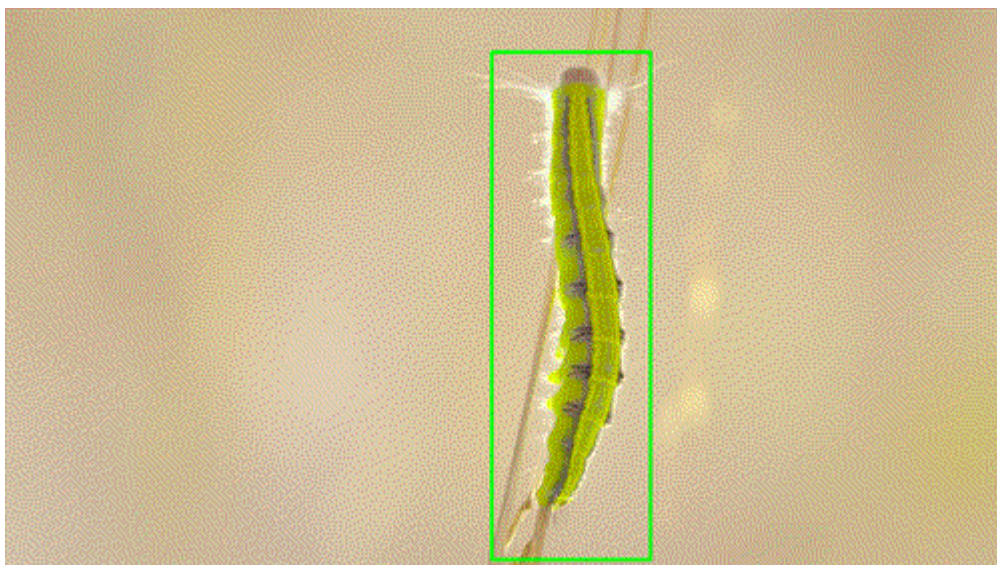
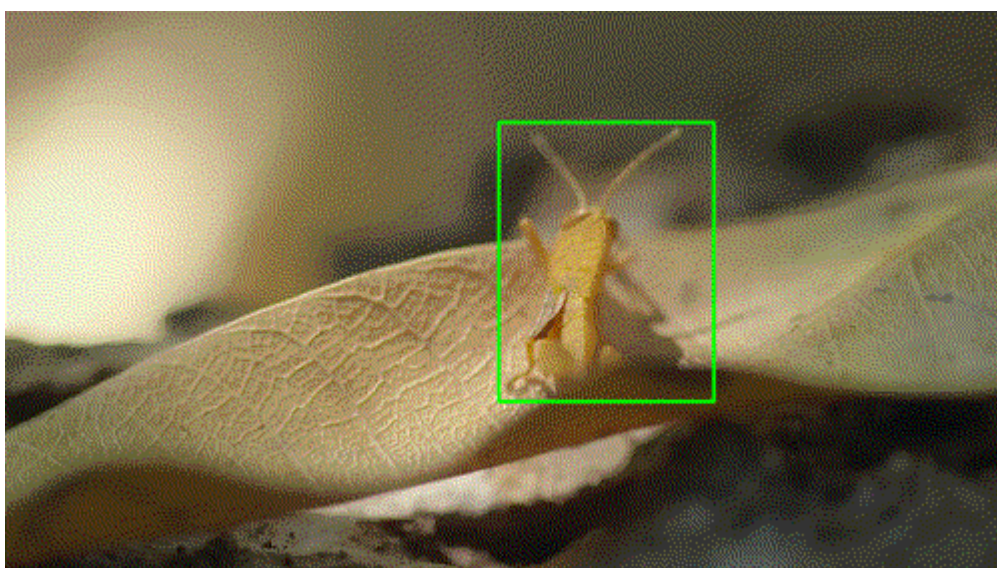
畫面應如下所示：

```

----- insect, confidence: 87%, frames: 25 -----
98.840 | (0.49327, 0.19617) | (0.69905, 0.69633)
98.960 | (0.49559, 0.19308) | (0.70631, 0.69671)
...
101.600 | (0.46668, 0.19776) | (0.76619, 0.69371)
101.720 | (0.46805, 0.20053) | (0.76447, 0.68703)
----- insect, confidence: 71%, frames: 23 -----
108.440 | (0.47343, 0.10694) | (0.63821, 0.98332)
108.560 | (0.46960, 0.10206) | (0.63033, 0.98285)
...
110.960 | (0.49466, 0.05102) | (0.65941, 0.99357)
111.080 | (0.49572, 0.04728) | (0.65762, 0.99868)

```

在對應影格上繪製定界框後，會得到以下結果：



摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行物件偵測和追蹤作業。進一步瞭解如何[偵測及追蹤物件](#)。

11. 偵測及追蹤標誌

您可以使用 Video Intelligence API 偵測及追蹤影片中的標誌。可偵測超過 10 萬個品牌和標誌。

將下列程式碼複製到 IPython 工作階段：

```
from datetime import timedelta
from typing import Optional, Sequence, cast

from google.cloud import videointelligence_v1 as vi

def detect_logos(
    video_uri: str, segments: Optional[Sequence[vi.VideoSegment]] = None
) -> vi.VideoAnnotationResults:
    video_client = vi.VideoIntelligenceServiceClient()
    features = [vi.Feature.LOGO_RECOGNITION]
    context = vi.VideoContext(segments=segments)
    request = vi.AnnotateVideoRequest(
        input_uri=video_uri,
        features=features,
        video_context=context,
    )

    print(f'Processing video "{video_uri}"...')
    operation = video_client.annotate_video(request)

    # Wait for operation to complete
    response = cast(vi.AnnotateVideoResponse, operation.result())
    # A single video is processed
    results = response.annotation_results[0]

    return results
```

請花點時間研究程式碼，瞭解如何使用 `annotate_video` 用戶端程式庫方法和 `LOGO_RECOGNITION` 參數分析影片及偵測標誌。

呼叫函式來分析影片的倒數第二個序列：

```
video_uri = "gs://cloud-samples-data/video/JaneGoodall.mp4"
segment = vi.VideoSegment(
    start_time_offset=timedelta(seconds=146),
    end_time_offset=timedelta(seconds=156),
)

results = detect_logos(video_uri, [segment])
```

等待影片處理完畢：

```
Processing video: "gs://cloud-samples-data/video/JaneGoodall.mp4"...
```

新增這個函式，列印偵測到的標誌清單：

```
def print_detected_logos(results: vi.VideoAnnotationResults):
    annotations = results.logo_recognition_annotations

    print(f" Detected logos: {len(annotations)} ".center(80, "-"))
    for annotation in annotations:
        entity = annotation.entity
        entity_id = entity.entity_id
        description = entity.description
        for track in annotation.tracks:
            confidence = track.confidence
            t1 = track.segment.start_time_offset.total_seconds()
            t2 = track.segment.end_time_offset.total_seconds()
            logo_frames = len(track.timestamped_objects)
            print(
                f"{confidence:4.0%}",
                f"{t1:>7.3f}",
                f"{t2:>7.3f}",
                f"{logo_frames:>3} fr.",
                f"{entity_id:<15}",
                f"{description}",
                sep=" | ",
            )
```

呼叫函式：

```
print_detected_logos(results)
```

畫面應如下所示：

```
----- Detected logos: 1 -----
92% | 150.680 | 155.720 | 43 fr. | /m/055t58 | Google Maps
```

新增這個函式，列印偵測到的標誌影格和定界框清單：

```

def print_logo_frames(results: vi.VideoAnnotationResults, entity_id: str):
    def keep_annotation(annotation: vi.LogoRecognitionAnnotation) -> bool:
        return annotation.entity.entity_id == entity_id

    annotations = results.logo_recognition_annotations
    annotations = [a for a in annotations if keep_annotation(a)]
    for annotation in annotations:
        description = annotation.entity.description
        for track in annotation.tracks:
            confidence = track.confidence
            print(
                f" {description}, "
                f" confidence: {confidence:.0%}, "
                f" frames: {len(track.timestamped_objects)} ".center(80, "-")
            )
            for timestamped_object in track.timestamped_objects:
                t = timestamped_object.time_offset.total_seconds()
                box = timestamped_object.normalized_bounding_box
                print(
                    f"{t:>7.3f}",
                    f"({box.left:.5f}, {box.top:.5f})",
                    f"({box.right:.5f}, {box.bottom:.5f})",
                    sep=" | ",
                )

```

使用 Google 地圖標誌實體 ID 呼叫函式：

```

maps_entity_id = "/m/055t58"
print_logo_frames(results, maps_entity_id)

```

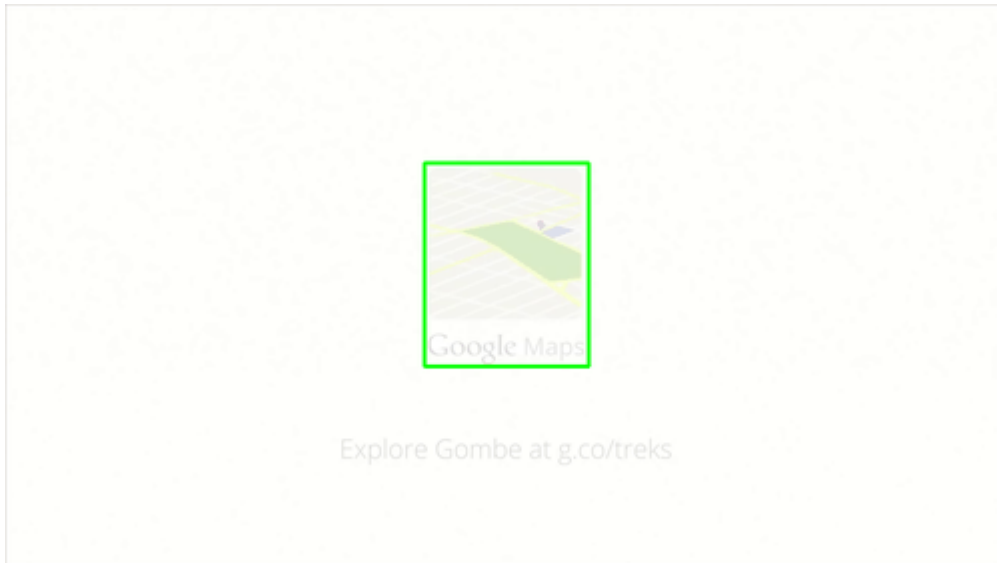
畫面應如下所示：

```

----- Google Maps, confidence: 92%, frames: 43 -----
150.680 | (0.42024, 0.28633) | (0.58192, 0.64220)
150.800 | (0.41713, 0.27822) | (0.58318, 0.63556)
...
155.600 | (0.41775, 0.27701) | (0.58372, 0.63986)
155.720 | (0.41688, 0.28005) | (0.58335, 0.63954)

```

在對應影格上繪製定界框後，會得到以下結果：



摘要

在這個步驟中，您已使用 Video Intelligence API 對影片執行標誌偵測和追蹤作業。進一步瞭解如何[偵測及追蹤標誌](#)。

12. 偵測多個特徵

您已瞭解如何逐一使用 Video Intelligence API 的各項功能，但您也可以偵測人物和臉部，並在單一要求中要求多項功能。

以下是您可以提出的要求類型，一次取得所有洞察資料：

```
from google.cloud import videointelligence_v1 as vi

video_client = vi.VideoIntelligenceServiceClient()
video_uri = "gs://..."
features = [
    vi.Feature.SHOT_CHANGE_DETECTION,
    vi.Feature.LABEL_DETECTION,
    vi.Feature.EXPLICIT_CONTENT_DETECTION,
    vi.Feature.SPEECH_TRANSCRIPTION,
    vi.Feature.TEXT_DETECTION,
    vi.Feature.OBJECT_TRACKING,
    vi.Feature.LOGO_RECOGNITION,
    vi.Feature.FACE_DETECTION, # NEW
    vi.Feature.PERSON_DETECTION, # NEW
]
context = vi.VideoContext(
    segments=...,
    shot_change_detection_config=...,
    label_detection_config=...,
    explicit_content_detection_config=...,
    speech_transcription_config=...,
    text_detection_config=...,
    object_tracking_config=...,
    face_detection_config=..., # NEW
    person_detection_config=..., # NEW
)
request = vi.AnnotateVideoRequest(
    input_uri=video_uri,
    features=features,
    video_context=context,
)

# video_client.annotate_video(request)
```

13. 恭喜！



您已學會如何使用 Python 呼叫 Video Intelligence API！

清除所用資源

如要清除開發環境，請在 Cloud Shell 中執行下列指令：

- 如果仍在 IPython 工作階段中，請返回 Shell：`exit`
- 停止使用 Python 虛擬環境：`deactivate`
- 刪除虛擬環境資料夾：`cd ~ ; rm -rf ./venv-videointel`

如要刪除 Google Cloud 專案，請在 Cloud Shell 中執行下列操作：

- 擷取目前的專案 ID：`PROJECT_ID=$(gcloud config get-value core/project)`
- 確認這是要刪除的專案：`echo $PROJECT_ID`
- 刪除專案：`gcloud projects delete $PROJECT_ID`

瞭解詳情

- 在瀏覽器中測試示範內容：<https://zackakil.github.io/video-intelligence-api-visualiser>
- Video Intelligence 說明文件：<https://cloud.google.com/video-intelligence/docs>
- Beta 版功能：<https://cloud.google.com/video-intelligence/docs/beta>
- Google Cloud 上的 Python：<https://cloud.google.com/python>
- Python 適用的 Cloud 用戶端程式庫：<https://github.com/googleapis/google-cloud-python>

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。
